DOKUMENTACJA PROJEKTOWA

[redacted]

System Wypożyczalni Sprzętu Wędkarskiego

Wersja: 1.1r
Data: 11-09-2026
Autor: Kacper Antoniak
[redacted]

[redacted]

Spis treści

1. Wymagania użytkownika 1
   1.1. Kontekst i tło 1
   1.2. Historyjka użytkownika 1
2. Diagram przypadków użycia 2
   2.1. Aktorzy systemu 2
   2.2. Diagram 2
3. Wymagania niefunkcjonalne 3
   3.1. Wydajność 3
   3.2. Bezpieczeństwo 3
   3.3. Niezawodność i dostępność 3
   3.4. Skalowalność 3
   3.5. Użyteczność 3
   3.6. Przenośność i kompatybilność 3
4. Diagram klas - analityczny 4
   4.1. Diagram 4
5. Diagram klas - projektowy 5
   5.1. Diagram 6
6. Scenariusz przypadku użycia 7
   6.1. Identyfikacja przypadku użycia 7
   6.2. Scenariusz główny 7
   6.3. Scenariusze alternatywne 7
   6.4. Scenariusze wyjątkowe 8
7. Diagram aktywności dla przypadku użycia 9
8. Diagram stanu dla klasy 10
   8.1. Wybrana klasa 10
   8.2. Diagram 11
9. Projekt GUI 12
   9.1. Makiety ekranów (mockupy) 12
      9.1.1. Ekran główny - Lista klientów / historia wypożyczeń 12
      9.1.2. Nowe wypożyczenie 13
      9.1.3. Panel podobnych ofert 13
      9.1.4. Szczegóły wypożyczenia / zmiana statusu 14
10. Omówienie decyzji projektowych i skutków analizy dynamicznej 14
   10.1. Kluczowe decyzje architektoniczne 14
   10.2. Decyzje dotyczące modelu danych 15
   10.3. Analiza dynamiczna - wnioski 15
   10.4. Wnioski końcowe 15
   10.5. Wykorzystane narzędzia 15

1. Wymagania użytkownika

1.1. Kontekst i tło

Coraz więcej wędkarzy, zarówno amatorów, jak i doświadczonych sportowców nie chce inwestować w kosztowny sprzęt na potrzeby jednorazowego wyjazdu lub próby nowej techniki połowu. Sklepy wędkarskie obsługują głównie sprzedaż, a oferta wypożyczalni jest rozproszona i zdarza się, że nie jest dostępna.

System Wypożyczalni Sprzętu Wędkarskiego ma za zadanie usprawnić proces rezerwacji i wypożyczania sprzętu (kołowrotków, wędek, szczytówek) poprzez scentralizowaną platformę obsługiwaną przez pracowników wypożyczalni.

Główny problem, który rozwiązuje system, to brak możliwości szybkiego sprawdzenia dostępności konkretnego zestawu sprzętu (np. kołowrotka z nawiniętą żyłką o określonej grubości) oraz brak mechanizmu sugestii podobnych ofert w przypadku niedostępności wybranej konfiguracji.

1.2. Historyjka użytkownika

Jako pracownik wypożyczalni chcę móc szybko zarejestrować klienta i założyć dla niego wypożyczenie dowolnego sprzętu (wędki, szczytówki, kołowrotka z żyłką) na określoną liczbę godzin, aby sprawnie obsłużyć wędkarza przy ladzie. W przypadku braku dokładnie takiej konfiguracji kołowrotka z żyłką, jaką życzy sobie klient chcę, aby system automatycznie zaproponował podobne dostępne produkty: ten sam model kołowrotka z inną grubością żyłki lub inny model kołowrotka z żyłką o poszukiwanej grubości. System powinien umożliwiać odnotowywanie zwrotów, przeglądanie historii wypożyczeń klientów oraz naliczanie dodatkowych opłat (kary za opóźnienie, opłaty za uszkodzenie).

Jako menadżer wypożyczalni chcę zarządzać katalogiem sprzętu (dodawać nowe egzemplarze, edytować istniejące, dezaktywować wycofane), aby oferta wypożyczalni była zawsze aktualna.

Punkty wymagań użytkownika:

1. System umożliwia rejestrację i logowanie pracowników.
2. System przechowuje dane klientów (imię, nazwisko, telefon, e-mail) wraz z ich adresem.
3. System umożliwia pracownikom przeglądanie katalogu sprzętu z podziałem na kategorie (kołowrotki, wędki, szczytówki).
4. Klient (za pośrednictwem pracownika) może wypożyczyć dowolny zestaw sprzętu (wędka, szczytówka, opcjonalnie kołowrotek z nawiniętą żyłką) na określoną liczbę godzin.
5. Jeżeli klient wybierze kołowrotek i poda konkretną grubość żyłki, a dokładna konfiguracja nie jest dostępna, system proponuje podobne oferty (ten sam kołowrotek z inną grubością żyłki lub inny model z tą samą grubością żyłki).
6. Klient decyduje się na ofertę alternatywną lub rezygnuje z kołowrotka.
7. Po zatwierdzeniu zamówienia system tworzy wypożyczenie z odpowiednimi pozycjami, wyliczoną kwotą oraz rejestruje płatność.
8. Pracownik odnotowuje zwrot sprzętu, a system zmienia status wypożyczenia na „Zwrócone". Pracownik może też naliczać dodatkowe opłaty (kara za opóźnienie, opłata za uszkodzenie).
9. System przechowuje dane trwale w relacyjnej bazie danych.

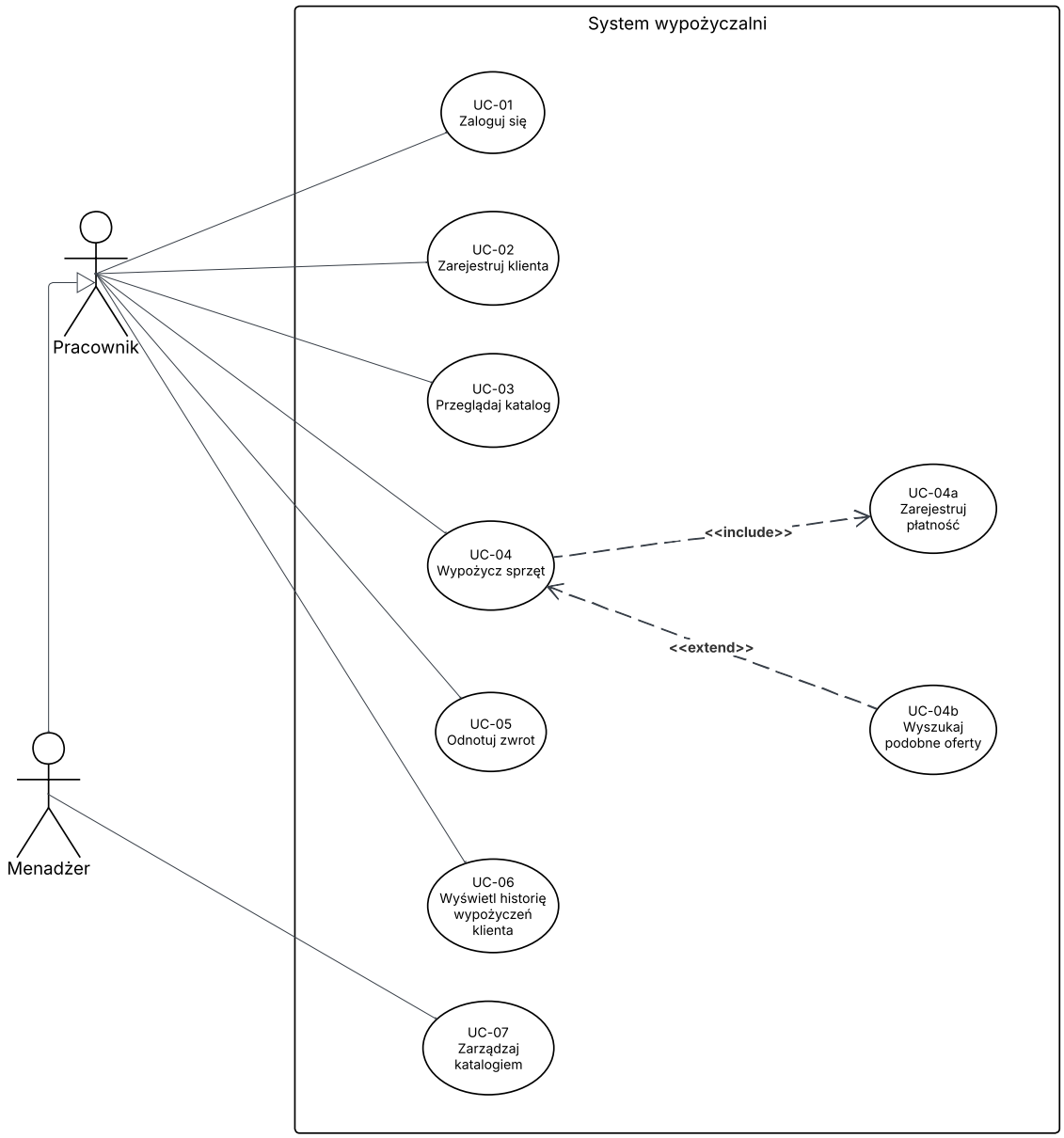
2. Diagram przypadków użycia

2.1. Aktorzy systemu

- **Pracownik** - obsługuje klientów przy stanowisku, zarządza stanem sprzętu, rejestruje wypożyczenia i zwroty.
- **Menadżer** (specjalizacja Pracownika) - posiada dodatkowe uprawnienia do zarządzania katalogiem sprzętu (dodawanie, edycja, usuwanie).

Klient jest interesariuszem pośrednim. Nie korzysta z systemu samodzielnie, lecz przekazuje swoje potrzeby pracownikowi, który wykonuje odpowiednie operacje w systemie.

2.2. Diagram



3. Wymagania niefunkcjonalne

3.1. Wydajność

- **WN-P-01**: System powinien obsługiwać co najmniej 20 jednoczesnych użytkowników bez odczuwalnego spadku wydajności.
- **WN-P-02**: Czas odpowiedzi dla typowej operacji (wyszukanie sprzętu, rejestracja wypożyczenia) nie powinien przekraczać 2 sekund przy standardowym połączeniu internetowym.
- **WN-P-03**: Zapytanie o podobne oferty (UC-04b) powinno zwracać wyniki w czasie poniżej 1 sekundy dla katalogu do 10 000 pozycji sprzętowych.

3.2. Bezpieczeństwo

- **WN-B-01**: Dostęp do panelu pracownika wymaga uwierzytelnienia (login + hasło); hasła będą zaszyfrowane przez bcrypt.
- **WN-B-02**: Komunikacja między frontendem Angular a backendem ASP.NET Core odbywa się wyłącznie przez HTTPS.

3.3. Niezawodność i dostępność

- **WN-N-01**: System powinien być dostępny przez co najmniej 99,0% czasu w ujęciu miesięcznym (dopuszczalny przestój: ~7 h/miesiąc).
- **WN-N-02**: W przypadku awarii bazy danych czas przywrócenia systemu (RTO) nie powinien przekraczać 30 minut.
- **WN-N-03**: Baza danych objęta codzienną automatyczną kopią zapasową.

3.4. Skalowalność

- **WN-S-01**: Architektura REST API umożliwia niezależne skalowanie warstwy backendowej bez zmian po stronie frontendu.
- **WN-S-02**: Model danych EF Core zaprojektowany z myślą o migracji do większej instancji SQL Server bez przebudowy schematu.

3.5. Użyteczność

- **WN-U-02**: Nowy pracownik powinien być w stanie samodzielnie zarejestrować wypożyczenie po 15-minutowym wprowadzeniu bez formalnego szkolenia.
- **WN-U-03**: Aplikacja responsywna, poprawne działanie na rozdzielczościach od 1024 x 768 px (stanowisko kasowe) do pełnego HD.

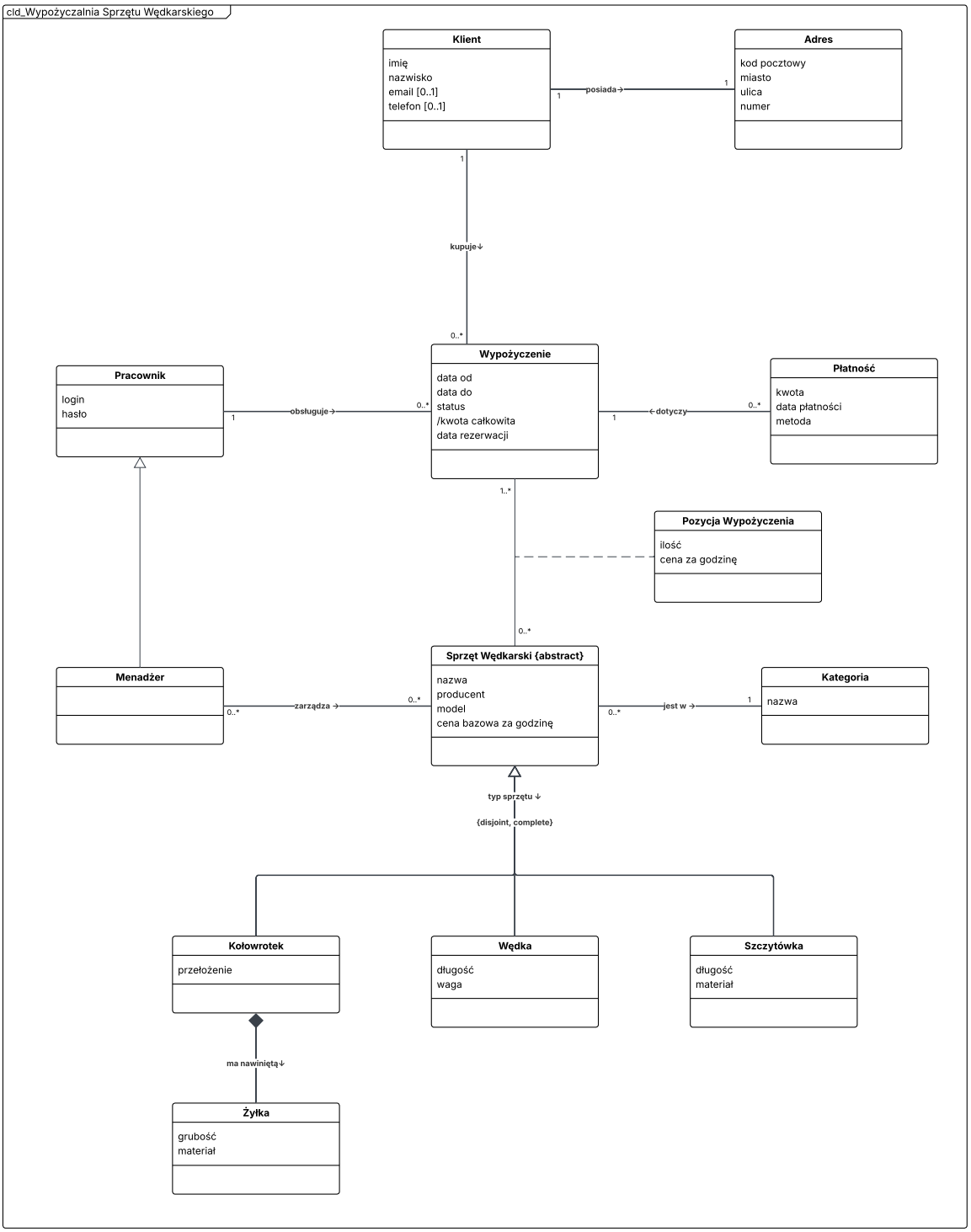
3.6. Przenośność i kompatybilność

- **WN-K-01**: Frontend powinien działać poprawnie w przeglądarkach: Chrome $\geq$ 120, Firefox $\geq$ 121, Edge $\geq$ 120.
- **WN-K-02**: Backend możliwy do uruchomienia na wielu platformach.
- **WN-K-03**: Dane przechowywane w SQL Server lub PostgreSQL. Zmiana silnika wymaga wyłącznie podmiany connection string i migracji EF Core.

4. Diagram klas - analityczny

Na diagramie analitycznym celowo nie umieszczono metod, ich źródłem jest analiza dynamiczna (diagramy aktywności i stanu), a ich pełna lista znajduje się dopiero na diagramie projektowym.

4.1. Diagram



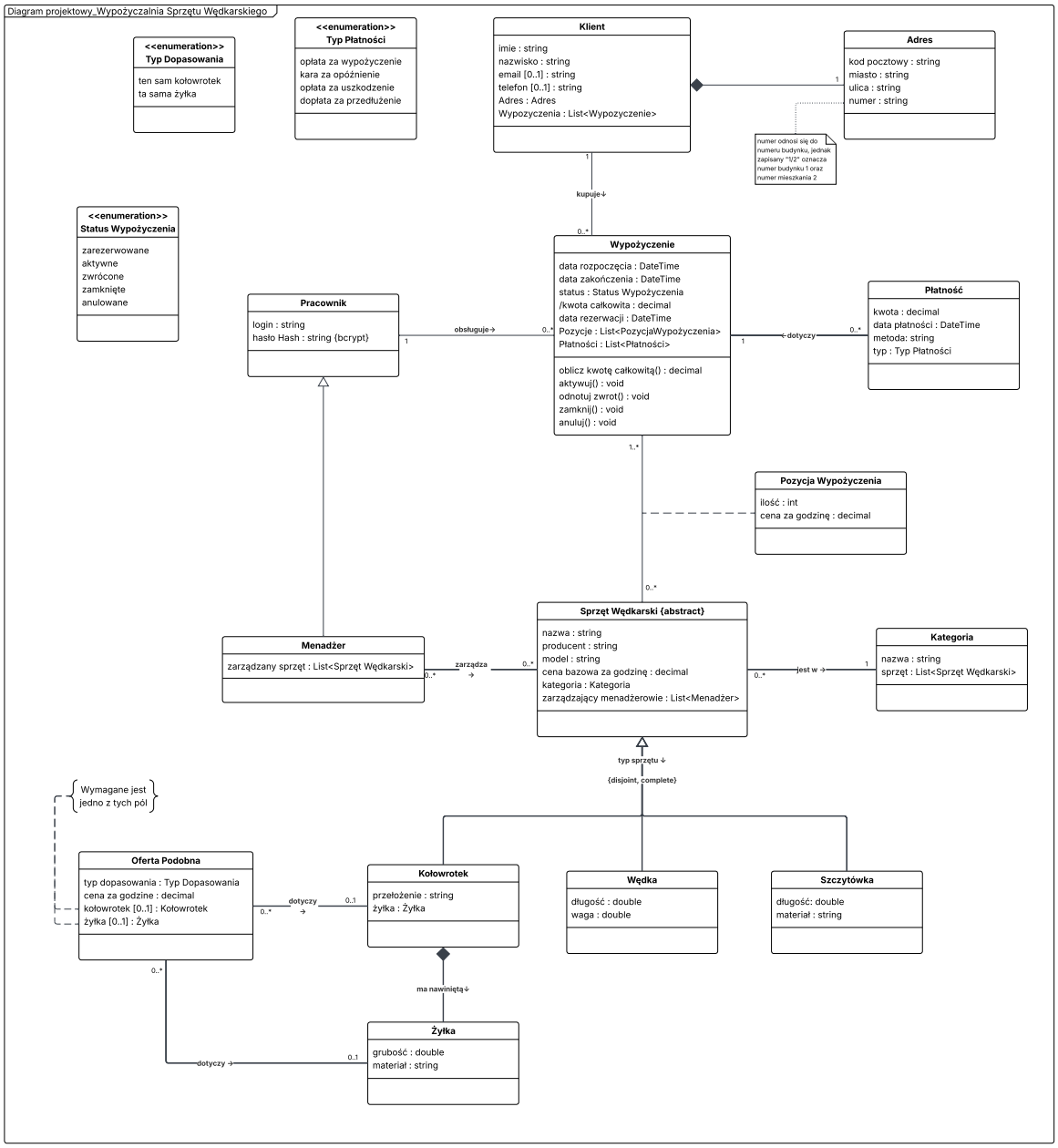
5. Diagram klas - projektowy

Diagram projektowy różni się od analitycznego następującymi elementami:

- Wszystkie atrybuty posiadają typy zgodne z C# (`string`, `int`, `decimal`, `DateTime`, `bool`, typy wyliczeniowe `enum`).
- Dodana klasa `Oferta Podobna` (wynik analizy dynamicznej) z atrybutem `typ dopasowania: Typ Dopasowania` i asocjacjami do `Kołowrotek` i `Żyłka`. Klasa nie jest persystowana, instancje tworzone w pamięci przez serwis `SugestieService`.
- Relacja `Kołowrotek` ⟶ `Żyłka` jest kompozycją (realizowaną przez `OwnsOne` w EF Core lub osobną tabelę z kluczem obcym, decyzja implementacyjna).
- Klasa `Wypożyczenie` posiada metody wynikające z diagramu stanu: `aktywuj()`, `odnotujZwrot()`, `zamknij()`, `anuluj()` oraz metodę `oblicz kwotę calkowitą(): decimal` dodaną na skutek analizy dynamicznej.
- Klasa `Płatność` posiada enum `Typ Płatności` rozróżniający rodzaje opłat.
- Wszystkie asocjacje z diagramu analitycznego mają swoje odzwierciedlenie w postaci kolekcji nawigacyjnych (np. `Klient.Wypozyczenia: List<Wypozyczenie>`).

Na diagramie pominięto techniczne atrybuty kluczy głównych (Id). Są one oczywistym elementem każdej encji i nie wnoszą informacji biznesowej. Z tego samego powodu pominięto gettery oraz settery, które będą niezbędne dopiero na etapie implementacji.

5.1. Diagram


Diagram projektowy_Wypożyczalnia Sprzętu Wędkarskiego
<<enumeration>>
Typ Dopasowania
ten sam kołowrotek
ta sama żyłka
<<enumeration>>
Typ Płatności
opłata za wypożyczenie
kara za opóźnienie
opłata za uszkodzenie
dopłata za przedłużenie
Klient
imie : string
nazwisko : string
email [0..1] : string
telefon [0..1] : string
Adres : Adres
Wypozyczenia : List<Wypozyczenie>
Adres
kod pocztowy : string
miasto : string
ulica : string
numer : string
numer odnosi się do numeru budynku, jednak zapisany "1/2" oznacza numer budynku 1 oraz numer mieszkania 2
kupuje
<<enumeration>>
Status Wypożyczenia
zarezerwowane
aktywne
zwrócone
zamknięte
anulowane
Wypożyczenie
data rozpoczęcia : DateTime
data zakończenia : DateTime
status : Status Wypożyczenia
/kwota całkowita : decimal
data rezerwacji : DateTime
Pozycje : List<PozycjaWypożyczenia>
Płatności : List<Płatności>
oblicz kwotę całkowitą() : decimal
aktywuj() : void
odnotuj zwrot() : void
zamknij() : void
anuluj() : void
Pracownik
login : string
hasło Hash : string {bcrypt}
obsługuje
dotyczy
Płatność
kwota : decimal
data płatności : DateTime
metoda: string
typ : Typ Płatności
Pozycja Wypożyczenia
ilość : int
cena za godzinę : decimal
Menadżer
zarządzany sprzęt : List<Sprzęt Wędkarski>
zarządza
Sprzęt Wędkarski {abstract}
nazwa : string
producent : string
model : string
cena bazowa za godzinę : decimal
kategoria : Kategoria
zarządzający menadżerowie : List<Menadżer>
jest w
Kategoria
nazwa : string
sprzęt : List<Sprzęt Wędkarski>
typ sprzętu
{disjoint, complete}
Wymagane jest jedno z tych pól
Oferta Podobna
typ dopasowania : Typ Dopasowania
cena za godzine : decimal
kołowrotek [0..1] : Kołowrotek
żyłka [0..1] : Żyłka
dotyczy
Kołowrotek
przełożenie : string
żyłka : Żyłka
Wędka
długość : double
waga : double
Szczytówka
długość: double
materiał : string
ma nawiniętą
Żyłka
grubość : double
materiał : string
dotyczy

6. Scenariusz przypadku użycia

6.1. Identyfikacja przypadku użycia

Nazwa:	UC-04 - Wypożycz sprzęt
Identyfikator:	UC-04
Aktor inicjujący:	Pracownik
Aktorzy dodatkowi:	Brak - Klient jest interesariuszem pośrednim
Przypadki include:	UC-04a - Zarejestruj płatność
Przypadki extend:	UC-04b - Wyszukaj podobne oferty (warunek: brak żądanej konfiguracji kołowrotka)
Warunek wstępny:	Pracownik jest zalogowany; katalog sprzętu jest załadowany.
Warunek końcowy (sukces):	Wypożyczenie zostało zapisane w bazie; status = „Zarezerwowane” lub „Aktywne”; kwota obliczona; płatność zarejestrowana.
Warunek końcowy (porażka):	Wypożyczenie nie zostało utworzone; stan magazynowy niezmieniony.

6.2. Scenariusz główny

1. Pracownik znajduje się w widoku głównym i wpisuje nazwisko klienta w pole wyszukiwania.
2. System filtruje listę i wyświetla tylko pasujących klientów.
3. Pracownik klika wybranego klienta na liście, system ładuje jego dane i historię wypożyczeń w prawym panelu.
4. Pracownik klika przycisk „Nowe wypożyczenie” znajdujący się nad historią wypożyczeń.
5. System otwiera formularz nowego wypożyczenia z automatycznie uzupełnionymi danymi wybranego klienta.
6. Pracownik wybiera daty i godziny wypożyczenia (data rozpoczęcia $\geq$ bieżąca chwila).
7. Pracownik wybiera sprzęt- dowolną kombinację wędki, szczytówki i kołowrotka (co najmniej jedna pozycja).
8. [Jeśli wybrano kołowrotek] Klient podaje żądaną grubość żyłki.
9. System sprawdza dostępność wybranego sprzętu w żądanym terminie.
10. System wyświetla zestawienie sprzętu z cenami za godzinę i wyliczoną kwotą całkowitą.
11. Pracownik zatwierdza wypożyczenie.
12. System tworzy obiekt Wypożyczenie ze statusem „Zarezerwowane”.
13. System wykonuje UC-04a- rejestracja płatności (pracownik wybiera metodę, system zapisuje obiekt Płatność).
14. System wyświetla podsumowanie z numerem wypożyczenia.

6.3. Scenariusze alternatywne

Alt. 3a - Brak klienta w systemie:

3a.1. System informuje, że nie znaleziono pasujących klientów.

3a.2. Pracownik wybiera opcję „Zarejestruj nowego klienta” (UC-02).

3a.3. Po rejestracji system automatycznie wczytuje dane nowego klienta, powrót do kroku 4.

Alt. 8a - Brak dokładnej konfiguracji kołowrotka (nietrywialny):

8a.1. System stwierdza brak kołowrotka z żyłką o żądanej grubości.

8a.2. System wykonuje UC-04b - Wyszukaj podobne oferty:
- Wyszukuje: (a) ten sam model kołowrotka z dostępnymi grubościami żyłki, (b) inne modele kołowrotków z żyłką o żądanej grubości.
- Zwraca dwie listy `Oferta Podobna` z typem dopasowania i ceną za godzinę.

8a.3. Pracownik prezentuje klientowi listę alternatyw.

8a.4. Klient wybiera jedną z ofert → powrót do kroku 9 (zastąpienie kołowrotka alternatywą).

Alt. 8b - Klient rezygnuje z kołowrotka po zobaczeniu alternatyw:

8b.1. Klient nie akceptuje żadnej alternatywy.

8b.2. Pracownik usuwa kołowrotek z zamówienia; pozostałe pozycje zostają bez zmian.

8b.3. Powrót do kroku 9.

6.4. Scenariusze wyjątkowe

Wyjątek - data rozpoczęcia w przeszłości:

System odrzuca datę rozpoczęcia wcześniejszą niż bieżąca chwila i wyświetla komunikat. Pracownik musi poprawić datę, aby kontynuować.

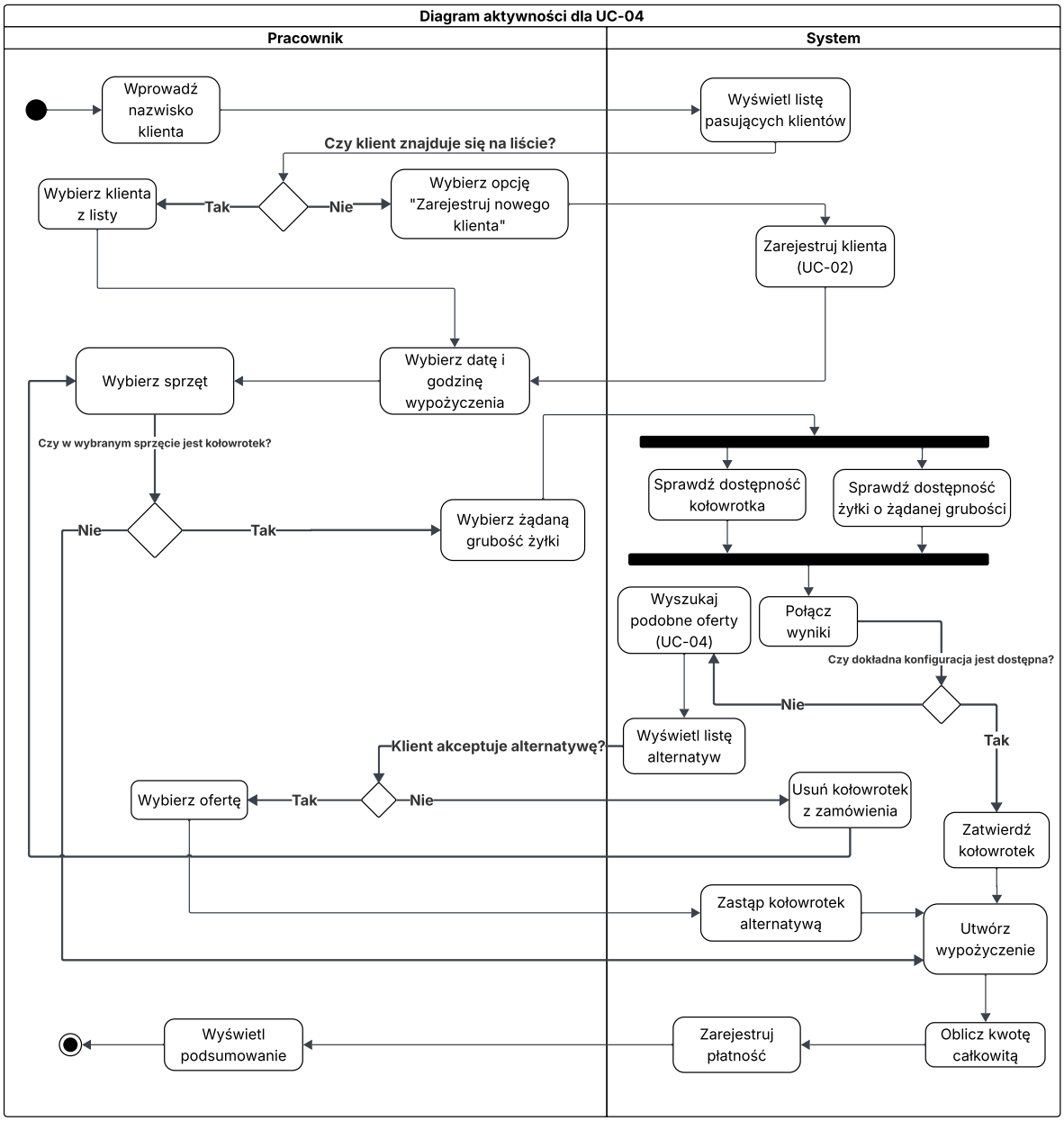
Wyjątek - brak dostępności zamówienia:

Jeśli którykolwiek z wybranych egzemplarzy (oprócz kołowrotka - obsłużonego przez UC-04b) jest niedostępny w żądanym terminie, system wyświetla komunikat i prosi o zmianę wyboru lub anulowanie.

Wyjątek - klient odmawia zapłaty za uszkodzenie:

Jeśli przy zwrocie zostanie stwierdzone uszkodzenie sprzętu, pracownik nalicza dodatkową opłatę przez UC-04a (rejestracja płatności) z typem „Opłata za uszkodzenie". Jeśli klient odmawia jej uiszczenia, wypożyczenie pozostaje w stanie „Zwrócone" z nieuregulowaną płatnością. System nie pozwala na zamknięcie wypożyczenia do czasu rozliczenia wszystkich należności. Taka sytuacja wymaga eskalacji do menedżera, który może podjąć decyzję o umorzeniu opłaty lub windykacji poza systemem.

7. Diagram aktywności dla przypadku użycia



8. Diagram stanu dla klasy

8.1. Wybrana klasa

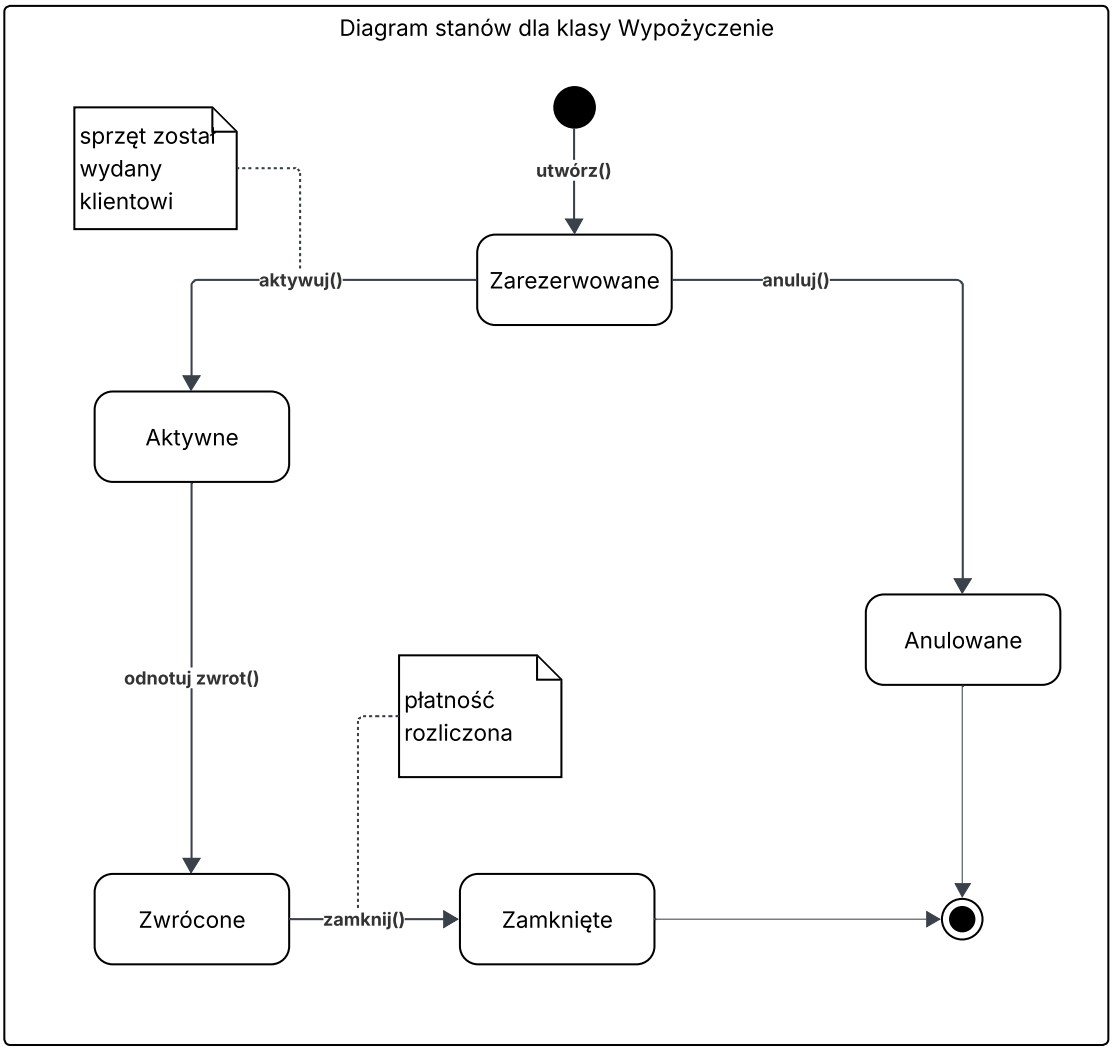
Wybrana klasa: **`Wypożyczenie`**

Klasa `Wypożyczenie` jest centralnym bytem systemu i przechodzi przez wyraźnie zdefiniowane stany w trakcie swojego cyklu życia - od momentu utworzenia (stan `Zarezerwowane`), przez aktywne wypożyczenie, aż do zamknięcia. Analiza stanów tej klasy bezpośrednio wpłynęła na atrybuty i metody widoczne na projektowym diagramie klas (pole `status: Status Wypożyczenia` typu enum oraz metody `aktywuj()`, `odnotujZwrot()`, `zamknij()`, `anuluj()`).

Przejścia między stanami:

- `[*]` → `Zarezerwowane`: utworzenie nowego wypożyczenia.
- `Zarezerwowane` → `Aktywne`: `aktywuj()` - sprzęt wydany klientowi.
- `Zarezerwowane` → `Anulowane`: `anuluj()` - anulowanie przed wydaniem.
- `Aktywne` → `Zwrócone`: `odnotujZwrot()` - klient oddał sprzęt.
- `Zwrócone` → `Zamknięte`: `zamknij()` - płatność rozliczona.

8.2. Diagram

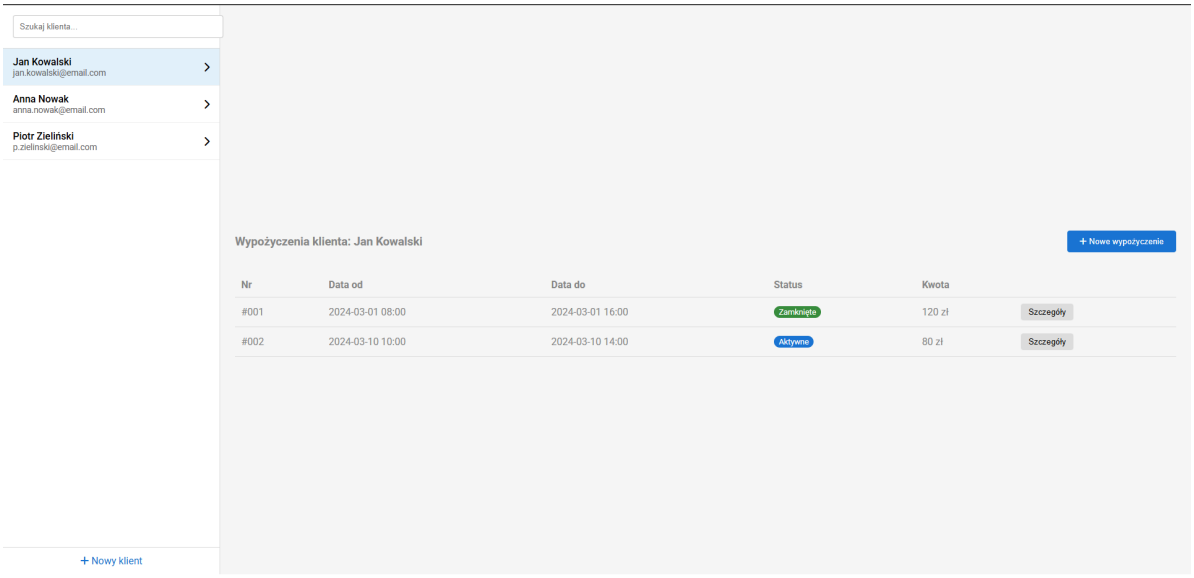

Diagram stanów dla klasy Wypożyczenie
sprzęt został wydany klientowi
utwórz()
aktywuj()
Zarezerwowane
anuluj()
Aktywne
Anulowane
odnotuj zwrot()
płatność rozliczona
Zwrócone
zamknij()
Zamknięte

9. Projekt GUI

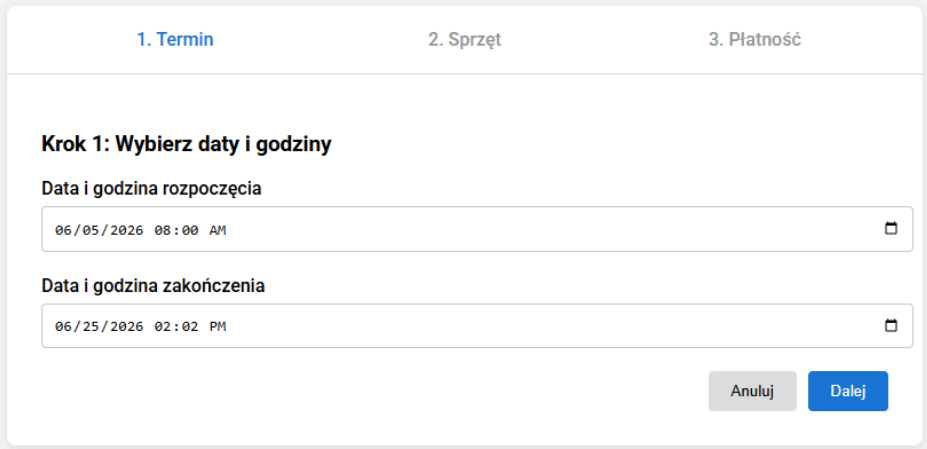
9.1. Makiety ekranów (mockupy)

Poniższe grafiki mają na celu przekazać wizję. Układ strony oraz niektóre elementy mogą się zmienić.

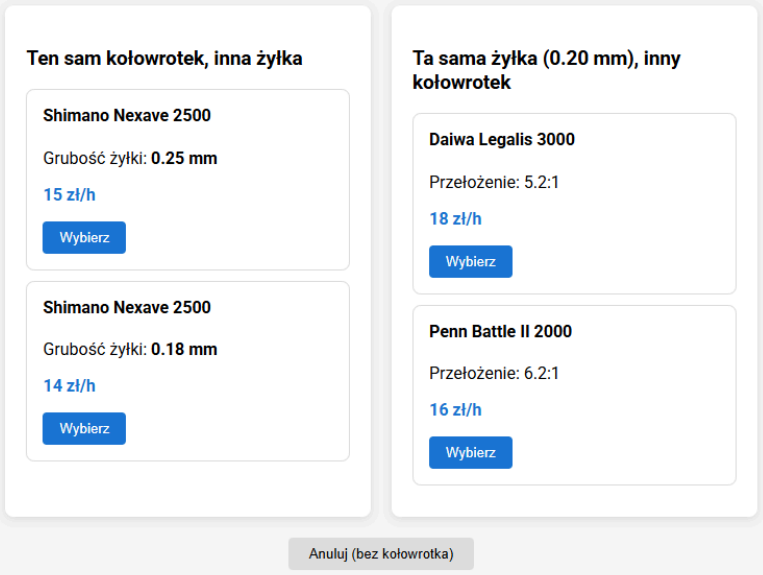
9.1.1. Ekran główny - Lista klientów / historia wypożyczeń



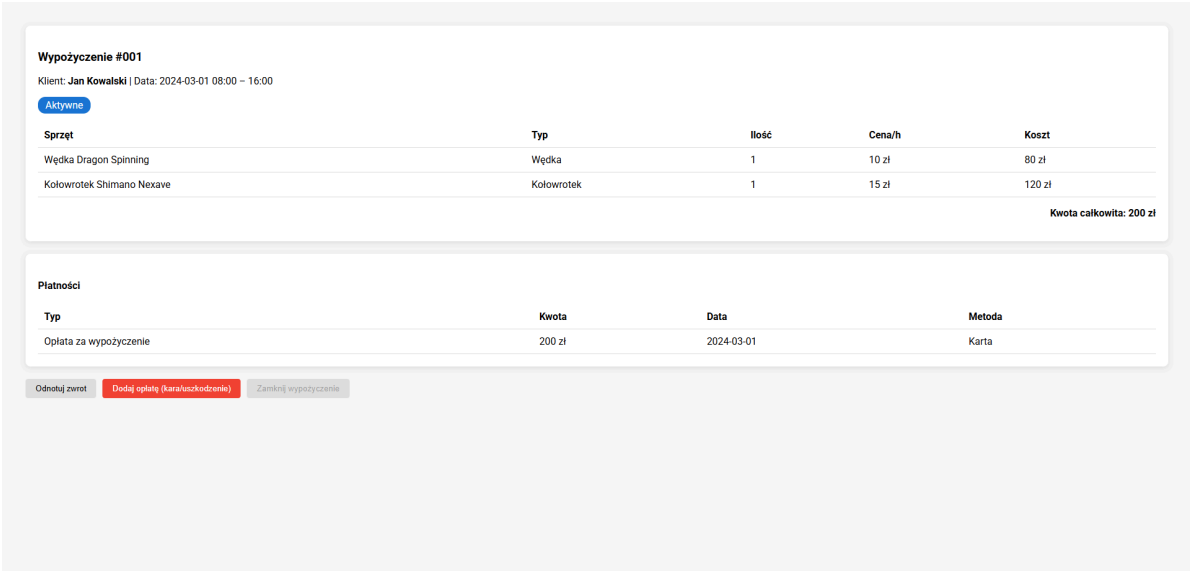
9.1.2. Nowe wypożyczenie



9.1.3. Panel podobnych ofert



9.1.4. Szczegóły wypożyczenia / zmiana statusu



10. Omówienie decyzji projektowych i skutków analizy dynamicznej

10.1. Kluczowe decyzje architektoniczne

Architektura REST API (ASP.NET Core) + Angular

Wybrałem architekturę rozdzielającą backend od frontendu. Backend udostępnia REST API zgodne z konwencjami HTTP (GET/POST/PUT/DELETE), a Angular komunikuje się przez serwisy HTTP. Uzasadnienie: ułatwia niezależne testowanie obu warstw, umożliwia późniejsze dodanie aplikacji mobilnej bez zmian backendu.

ORM Entity Framework Core (code-first) zamiast ręcznego SQL

Schemat bazy danych generowany z klas C# przez migracje EF Core.
Podejście to zmniejsza rozbieżność między modelem obiektowym a relacyjnym oraz przyspiesza development.

Rezygnacja z warstwy repozytoriów na rzecz bezpośredniego użycia DbContext w serwisach

W architekturze backendu każdy serwis aplikacyjny (np. `KlientService`, `WypozyczenieService`) otrzymuje wstrzyknięty `AppDbContext`. Logika dostępu do danych realizowana jest przez LINQ bezpośrednio na `DbSet`, z wykorzystaniem `Include` do ładowania powiązanych encji.

10.2. Decyzje dotyczące modelu danych

Realizacja dziedziczenia - Table-Per-Hierarchy (TPH)

Klasy `Kołowrotek`, `Wędka`, `Szczytówka` dziedziczące po `Sprzęt Wędkarski` zmapowane do jednej tabeli `SprzetyWedkarskie` z kolumną dyskryminatora `TypSprzetu` (varchar). Uzasadnienie: prostsze zapytania, mniejsza liczba joinów. Wadą są kolumny nullable dla atrybutów specyficznych podklas.

Realizacja kompozycji Kołowrotek - Żyłka:

W encji `Kołowrotek` zastosuję owned entity (`OwnsOne`) lub obowiązkowy klucz obcy `ZylkaId` (NOT NULL). Zapewnia to spójność z regułą biznesową: każdy kołowrotek ma nawiniętą żyłkę, która nie istnieje jako samodzielny byt w naszym systemie. Usunięcie kołowrotka kaskadowo usuwa przypisaną mu żyłkę.
Reguła ta wynika z tego, że nawijanie żyłki na kołowrotek jest czasochłonne a zdjęcie jej sprawia, że żyłka nie nadaje się do dalszego użytkowania.

Realizacja klasy asocjacji `Pozycja Wypożyczenia`:

`Pozycja Wypożyczenia` jest klasą asocjacji między klasą `Wypożyczenie` a klasą `Sprzęt Wędkarski`, przechowującą atrybuty samego powiązania (`ilość`, `cena za godzinę`). W EF Core realizowana jako zwykła encja z dwoma kluczami obcymi.

Realizacja klasy `Oferta Podobna`:

Klasa wynikła z analizy dynamicznej (diagram aktywności, gałąź UC-04b). Nie jest persystowana w bazie, instancje tworzone w pamięci przez serwis `SugestieService` na podstawie zapytania LINQ do tabel `Koloworotki` i `Zylki`. Atrybut `typd opasowania` (enum: `ten sam kolowrotek`, `TaSamaZylka`) pojawia się na projektowym diagramie klas jako skutek analizy.

10.3. Analiza dynamiczna - wnioski

Przeprowadzenie analizy dynamicznej ujawniło następujące skutki implementacyjne widoczne na projektowym diagramie klas:

- Dodana metoda `oblicz kwotę całkowitą(): decimal` w klasie `Wypożyczenie`.
- Dodany atrybut `status: Status Wypożyczenia` jako enum.
- Dodane metody `aktywuj()`, `odnotujZwrot()`, `zamknij()`, `anuluj()` w klasie `Wypożyczenie`.
- Dodana klasa `Oferta Podobna` z asocjacjami do `Kołowrotek` i `Żyłka`.
- Dodany warunek w metodzie `zamknij()` blokujący zamknięcie wypożyczenia przy nieuregulowanych płatnościach (wynika ze scenariusza wyjątkowego o odmowie zapłaty).

10.4. Wnioski końcowe

Na etapie analizy i projektowania zdefiniowano pełny model dziedziny dla systemu wypożyczalni sprzętu wędkarskiego: 13 klas biznesowych, hierarchię dziedziczenia, nietrywialny przypadek użycia UC-04 z mechanizmem sugestii alternatyw oraz scenariusze alternatywne i wyjątkowe.

Analiza dynamiczna (diagram aktywności i diagram stanu klasy `Wypożyczenie`) pozwoliła uzupełnić projektowy diagram klas o metody i atrybuty niewidoczne na etapie analizy statycznej.

10.5. Wykorzystane narzędzia

Diagramowanie: Lucidchart
Dokumentacja: Typst